PH3120

Computational Physics Laboratory 1

CPL105 – Root Finding Lab Sheet

1. I. (a) IVT is a fundamental result in calculus that ensures the existence of a root within a continuous interval under certain conditions.
   If *f* is continuous function on a closed interval [a, b], and $f(a) \neq f(b)$, then for any N between $f(a)$ and $f(b)$, there exists c ∈ (a, b) such that $f(c)$ = N.

   (b) The IVT ensures that if $f(a).\ f(b) < 0$, then there is at least one root in the interval (a, b). The bisection method finds the root by repeatedly applying the IVT to subintervals.

   - Select two points a and b such that $f(a)\cdot f(b) < 0$. The, we can confirm there is at least one root in the interval [a, b] by the IVT.
   - Then calculate the midpoint.
   - Evaluate function at the midpoint
   - Then we can check for roots:
     - If $f(c) = 0$ or $|b-a|$ is sufficiently small (below a predefined tolerance ϵ), then c is the root.
     - If $f(a)\cdot f(c) < 0$, set b = c (the root is in the left subinterval).
     - If $f(\mathrm{b})\cdot f(c) < 0$, set a = c (the root is in the right subinterval).

   (c) $y(\gamma).\ y(\delta) < 0$

   If these two conditions are met, the Intermediate Value Theorem guarantees that there is at least one root (where y(x)=0) within the interval (γ,δ).

   (d) When we have β=(γ+δ)/2, this β is a midpoint of the interval [γ,δ].

   (e) The bisection method, though simple and consistently reliable for finding roots, has important practical considerations and limitations. A key requirement is finding an initial interval where the function's sign changes, which can be tricky for complex functions. While it guarantees convergence, its linear convergence rate is relatively slow compared to other methods, often requiring many iterations for high precision. Furthermore, it cannot find multiple roots within an interval or "even" roots where the function touches but doesn't cross the x-axis. It also doesn't utilize information from the function's derivative, limiting its efficiency. Despite these drawbacks, its robustness makes it a valuable foundational tool in numerical analysis.

II.

```
def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
       print("The function must have opposite signs at a and b.")

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c

        if f(a) * f(c) < 0:
            b = c
        else:
            a = c

    return (a + b) / 2
```

III. (a)

```
import numpy as np

def f(x):
    return x**2 - np.exp(-x) - 2 * np.sin(x)

def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("The function must have opposite signs at a and b.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2


root = bisection(f, 0, 2, 1e-6)
print("Root of f(x): ", root)
```

```
Root of f(x):  1.489588737487793
```

(b)

```
import numpy as np

def f(t):
    return 10 * t - 4.9 * t**2

def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("Bisection method fails. f(a) and f(b) must have opposite signs.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        elif f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2

root = bisection(f, 1, 100, 1e-6)
print("Time take projectile returns to initial height:", root, "seconds")
```

```
Time when projectile returns to initial height: 2.040816044807434 seconds
```

(c)

```
import numpy as np

def f(x):
    return x**2 - 3

def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("Bisection method fails. f(a) and f(b) must have opposite signs.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        elif f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2

root = bisection(f, 1, 2, 1e-6)
print("Square root of 3 is:", root)
```

```
Square root of 3 is: 1.732050895690918
```

(d)

```python
import numpy as np

def f(L):
    return L - (100*9.8)/(4*np.pi*np.pi)

def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("Bisection method fails. f(a) and f(b) must have opposite signs.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        elif f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2

root = bisection(f, 1, 100, 1e-6)
print("Length of the pendulum for T=10 :", root, "meters")
```

```
Length L for T=10 : 24.82368966192007 meters
```

(e)

```python
import numpy as np

def f(t):
    return 5 * (1 - np.exp(-t / 10)) - 2.5

def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("f(a) and f(b) must have opposite signs.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        elif f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2

time = bisection(f, 0, 50, 1e-6)
print("Voltage across the capacitor :", time, "V")
```

```
Voltage across the capacitor : 6.931471079587936 V
```

2. I.

(a) To update the approximation in each iteration of Newton-Raphson method, the following formula is used

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n}$$

(b)

Newton-Raphson's convergence heavily depends on the initial guess. A good guess near the root ensures fast convergence. However, a poor guess, or one in an area with multiple roots, can cause the method to diverge, oscillate, or find an unintended root.

(b) The Newton-Raphson method uses a function's derivative to find the tangent line at the current approximation. This tangent line's intersection with the x-axis provides the next, improved root estimate. This iterative process is repeated to refine the root until (desired accuracy is achieved.

(c) The Newton -Raphson method may fail to converge or converge slowly under the following condition. If the initial guess is too far from the root or in a region where the function has no root. If f'(xn) = 0 at any iteration, causing division by zero. If the function has a horizontal inflection point or saddle point near the initial guess, leading to poor tangent line behavior.

II.

```
def new_raphson(f , deriv_f, initial_guess, tolerance=1e-6, max_iteration=100):
    x = initial_guess
    for i in range(max_iteration):
        f_x = f(x)
        deriv_x = deriv_f(x)

        if deriv_x == 0:
            print("Derivative is zero. Newton-Raphson method cannot proceed.")
            return None

        x_new = x - f_x / deriv_x

        if abs(x_new - x) < tolerance:
            return x_new

        x = x_new

    Print("Maximum iteration reached. Method may not have converged.")
    return x
```

III. (a)

```
import numpy as np

def f(x):
    return x**2 - np.exp(-x) - 2 * np.sin(x)

def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("The function must have opposite signs at a and b.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2


root = bisection(f, 0, 2, 1e-6)
print("Root of f(x): ", root)
```

```
Root of f(x):  1.489588737487793
```

(b)

```python
import numpy as np
def f(t):
    return 10 * t - 4.9 * t**2

def f_prime(t):
    return 10 - 9.8*t

def new_raphson(f , deriv_f, initial_guess, tolerance=1e-6, max_iteration=100):
    t = initial_guess
    for i in range(max_iteration):
        f_t = f(t)
        deriv_t = deriv_f(t)

        if deriv_t == 0:
            print("Derivative is zero. Newton-Raphson method cannot proceed.")
            return None

        t_new = t - f_t / deriv_t

        if abs(t_new - t) < tolerance:
            return t_new

        t = t_new

    Print("Maximum iteration reached. Method may not have converged.")
    return t

Root = new_raphson(f , f_prime, initial_guess)

if Root is not None:
    print("Time take projectile returns to initial height: ",Root,"Seconds")
```

```
Time take projectile returns to initial height:  2.040816326530625 Seconds
```

(d)

```python
import numpy as np
def f(L):
    return L - (100*9.8)/(4*np.pi*np.pi)

def deriv_f(L):
    return 1

def new_raphson(f , deriv_f, initial_guess, tolerance=1e-6, max_iteration=100):
    L = initial_guess
    for i in range(max_iteration):
        f_L = f(L)
        deriv_L = deriv_f(L)

        if deriv_L == 0:
            print("Derivative is zero. Newton-Raphson method cannot proceed.")
            return None

        L_new = L - f_L / deriv_L

        if abs(L_new - L) < tolerance:
            return L_new

        L = L_new

    Print("Maximum iteration reached. Method may not have converged.")
    return L

Root = new_raphson(f , deriv_f, 20)

if Root is not None:
    print("Length of the pendulum for T=10 :", Root, "meters")
```

```
Length of the pendulum for T=10 : 24.82368999237276 meters
```

(e)

```python
import numpy as np
def f(t):
    return 5 * (1 - np.exp(-t / 10)) - 2.5

def f_prime(t):
    return np.exp(-t / 10)/2

def new_raphson(f , deriv_f, initial_guess, tolerance=1e-6, max_iteration=100):
    t = initial_guess
    for i in range(max_iteration):
        f_t = f(t)
        deriv_t = deriv_f(t)

        if deriv_t == 0:
            print("Derivative is zero. Newton-Raphson method cannot proceed.")
            return None

        t_new = t - f_t / deriv_t

        if abs(t_new - t) < tolerance:
            return t_new

        t = t_new

    Print("Maximum iteration reached. Method may not have converged.")
    return t

Root = new_raphson(f , f_prime, 10)

if Root is not None:
    print("Voltage across the capacitor :",Root,"V")
```

```
Voltage across the capacitor : 6.931471805599453 V
```

3. I.

(a)

```
import numpy as np
from scipy.optimize import fsolve

def f(x):
    return x**2 - np.exp(-x) - 2 * np.sin(x)

initial_guess = 1.5
root_found = fsolve(f, initial_guess)

print("The root found is : ",root_found)
```

```
The root found is :  [1.48958864]
```

(b)

```
import numpy as np
from scipy.optimize import fsolve

def f(t):
    return 10 * t - 4.9 * t**2

initial_guess = 1.5
root_found = fsolve(f, initial_guess)

print("Time take projectile returns to initial height: ",root_found,"Seconds")
```

```
Time take projectile returns to initial height:  [2.04081633] Seconds
```

(c)

```
import numpy as np
from scipy.optimize import fsolve

def f(x):
    return x**2 - 3

initial_guess = 1.5
root_found = fsolve(f, initial_guess)

print("The Square root of 3 is: ",root_found)
```

```
The Square root of 3 is:  [1.73205081]
```

(d)

```
import numpy as np
from scipy.optimize import fsolve

def f(L):
    return L - (100*9.8)/(4*np.pi*np.pi)

initial_guess = 1.5
root_found = fsolve(f, initial_guess)

print("Length of the pendulum for T=10 : ",root_found,"meters")
```

```
Length of the pendulum for T=10 :  [24.82368999] meters
```

(e)

```
import numpy as np
from scipy.optimize import fsolve

def f(t):
    return 5 * (1 - np.exp(-t / 10)) - 2.5

initial_guess = 1.5
root_found = fsolve(f, initial_guess)

print("Voltage across the capacitor : ",root_found,"V")
```

```
Voltage across the capacitor :  [6.93147181] V
```